Subject: FAP201

No	Name	Student ID
1	Phạm Minh Tùng	SE204386
2	Trần Tuấn Lâm	SE194892
3	Mai Đình Tuấn	SE204186
4	Bùi Lê Văn Khoa	SE203529

Comparative Study of Adder-Tree and Systolic-Array Accumulator Architectures for a Real-Time 3×3 Image Convolution Processor on FPGA

Abstract

This paper presents the design and experimental evaluation of a real-time 3×3 image convolution processor on FPGA, implementing two common image filters — sharpening and blurring. The main contribution is a quantitative comparison between two alternative hardware architectures for the accumulation stage of the convolution core: (1) an Adder Tree architecture, which sums all nine multiply-accumulate terms combinationally in a single clock cycle, and (2) a Systolic Array architecture, which decomposes the accumulation into a chain of nine Processing Elements (PEs), each performing exactly one multiply and one add per clock cycle. Both architectures were implemented in Verilog HDL, verified against an independent Python/NumPy golden reference model, and synthesized on Xilinx Vivado at three image resolutions (64x64, 128x128, 256x256). Experimental results show that the maximum operating frequency (Fmax) of the two architectures is nearly identical at every image size, while the Systolic Array consumes substantially more LUTs and flip-flops — a phenomenon explained by the fact that the true critical path of the design lies in the final blur-scaling stage, shared unchanged between both architectures, rather than in the accumulation stage that the Systolic Array was designed to optimize. The paper further shows that the relative resource overhead of the Systolic Array shrinks as image size grows, since the fixed cost of its timing-alignment shift-registers is increasingly diluted by the shared line-buffer cost, which scales with image width.

Keywords: FPGA, 3x3 convolution, systolic array, adder tree, real-time image processing, Verilog HDL, hardware architecture comparison.

I. Introduction

Convolution — multiplying each element of a sliding window by a kernel coefficient and summing the results into a single output pixel — is a foundational operation in both classical image processing (blurring, sharpening, edge detection) and modern convolutional neural networks (CNNs). Because this operation is repeated millions of times per image, hardware acceleration — particularly on FPGAs, owing to their reconfigurability and high degree of parallelism — has been studied extensively [1], [2].

A key design decision when implementing convolution in hardware is how the accumulation stage — the stage that sums the multiply results from every position in the kernel window — is organized. Two classical approaches are: (1) a combinational adder tree, which sums all terms within a single clock cycle using multi-level combinational logic, and (2) a systolic array, a concept introduced by Kung and Leiserson between 1978 and 1982 [3], [4], in which the computation is decomposed into a chain of synchronized Processing Elements (PEs), each performing a small part of the computation and forwarding a partial result to the next PE. The systolic architecture has been shown to deliver superior performance in large-scale accelerators such as Google's Tensor Processing Unit (TPU) [5], and remains an active research topic for CNN accelerators on FPGA [6]-[11].

However, most existing studies focus on large-scale two-dimensional (2D) systolic arrays for the convolutional layers of deep CNNs, where the number of channels and kernel size are large enough that the benefits of pipelining clearly outweigh the additional hardware cost. For a 3x3

kernel — the most common kernel size in classical image processing, and also the most widely used kernel size in modern CNNs [12] — whether a systolic architecture still provides a real benefit over a simpler adder tree at this small scale has not been clearly answered with concrete experimental evidence.

This paper contributes: (1) a complete Verilog HDL implementation of both architectures — Adder Tree and a one-dimensional (1D) chain Systolic Array — for the same 3x3 convolution core, sharing a common line buffer and window-generation front end; (2) a verification workflow using an independent Python golden model, which detected and helped fix a genuine timing bug during the merging of the two original filter modules; (3) experimental resource (LUT, FF, DSP) and maximum operating frequency (Fmax) data for both architectures, measured on Xilinx Vivado at three image resolutions; and (4) an analysis explaining why the theoretical benefit of the Systolic Array is not yet visible at 3x3 kernel scale, together with a well-founded prediction of the conditions under which that benefit should emerge.

II. Related Work

A. Systolic Array Architectures

The systolic array concept was first introduced by Kung in the foundational paper "Why Systolic Architectures?" [3], which describes a systolic system as a network of homogeneous processors through which data flows rhythmically, in a manner analogous to the pumping rhythm of a heart. Kung and Picard later proposed a one-dimensional systolic array specifically for multidimensional convolution [13], with the important property that the array's I/O bandwidth is independent of kernel size — this is precisely the architectural principle underlying the Systolic Array design in this paper. At industrial scale, Jouppi et al. describe Google's TPU [5], which uses a 256x256 two-dimensional systolic array of multiply-accumulate units for matrix multiplication in neural network inference, achieving substantially higher performance than contemporary CPUs and GPUs.

B. Systolic-Array-Based CNN Accelerators on FPGA

Several recent works apply systolic architectures to CNN accelerators on FPGA. Dua et al. propose Systolic-CNN [6], a one-dimensional systolic architecture defined in OpenCL, optimized for scalability and run-time reconfigurability across multiple CNN models. Related work presents a tiled systolic array generator for convolution kernels [7]. WinoCNN by Liu et al. [8] combines the Winograd algorithm with a systolic array to optimize DSP efficiency. Wan et al. propose a Deformable Systolic Array (DSA-CNN) with configurable Processing Elements [9]. Zhang et al. specifically study the problem of improving the operating frequency of systolic-array-based CNNs on FPGA [10], a topic directly relevant to the Fmax analysis in this

paper. More recently, systolic architectures have been extended to Transformer models [11], [14], demonstrating the flexibility of this computational model beyond CNNs alone.

C. Streaming Image-Processing Architectures on FPGA

The line-buffer-plus-sliding-window architecture is the standard method for implementing streaming convolution on FPGA, enabling pixel-by-pixel processing without external memory for the entire image [15]-[19]. Shi et al. present a high-throughput line-buffer architecture supporting arbitrary image sizes at run time [19]. Work on the hls4ml FPGA implementation also highlights the resource advantage of a line-buffer (shift-register-based) architecture over a full sliding-window-replication architecture [18] — this is precisely the design principle behind the line_buffer module used in this work.

D. Adder Trees and Multi-Operand Adders

From a digital-circuit-theory standpoint, adder trees and carry-save adders (CSA) are classical techniques for summing multiple operands with combinational delay of O(log n) rather than O(n) as with sequential addition [20], [21]. Prior work has analyzed FPGA logic-cell architectures specifically for efficient compressor-tree implementation [22], directly relevant to how Vivado synthesizes the adder tree in this design.

E. Sharpening and Blurring Filters

Unsharp masking — sharpening an image by adding back a fraction of the difference between the original image and a blurred version of it — is a classical sharpening technique in digital image processing [23], [24], described in detail in the standard textbook by Gonzalez and Woods [25]. The 3x3 sharpening kernel used in this paper (center coefficient 5, four orthogonal neighbors at -1) is a common discrete approximation of a Laplacian-based unsharp mask.

F. Pipelining, Retiming, and FPGA Timing Closure

Pipelining and retiming are fundamental techniques for achieving timing closure on FPGA, where pipelining breaks a long combinational logic path into multiple shorter register stages to increase the maximum operating frequency [26], [27]. This paper applies exactly this principle in the Systolic Array design — which is, in essence, a deep pipelining of the accumulation stage — and the experimental data show that the benefit of this technique depends strongly on whether the original critical path actually lies in the pipelined segment, a finding consistent with general guidance in the timing-closure literature [26].

G. RTL Verification with Golden Reference Models

Comparing the output of an RTL design against an independent, higher-abstraction-level reference model (a golden reference model, typically written in Python, MATLAB, or C++) is a

standard functional verification method in the semiconductor industry [28]-[30]. This paper applies this method using a Python/NumPy golden model that precisely mirrors the RTL's fixed-point arithmetic, including the approximate division-by-9 used in the blur filter.

III. System Architecture

A. Overall Pipeline

The system processes a grayscale image through a real-time streaming pipeline consisting of four main stages: (1) line_buffer — stores the two previous image rows using shift registers; (2) window_3x3 — combines the line-buffer data with the current pixel to form a 3x3 sliding window (nine pixels: p11 through p33); (3) the convolution core — multiplies each pixel by its corresponding kernel coefficient (selected by the mode signal: 0 = sharpen, 1 = blur) and accumulates the result into a single output pixel; (4) the output stage — synchronizes data_valid_out with o_pixel. Stage (3) is the only stage that differs between the two architectures compared in this paper; stages (1), (2), and (4) are identical, allowing the convolution core to be swapped without affecting the rest of the system.

B. Adder Tree Architecture

In the Adder Tree architecture (module conv_adder_tree), all nine multiplications (pixel x kernel coefficient) are computed in parallel and registered in a single clock cycle. In the following cycle, all nine multiply results are summed by a combinational adder tree (four addition levels, corresponding to log2(9)) into a single total. In the third cycle, this total is processed by the final stage: for sharpen, the value is clipped to the range [0, 255]; for blur, the value is multiplied by the constant 114 and right-shifted by 10 bits (an approximation of division by 9 using multiply-and-shift, avoiding a true division operation, which is expensive on FPGA). The total pipeline latency of this architecture is 3 clock cycles.

C. Systolic Array Architecture

In the Systolic Array architecture (module conv_systolic), the nine-term accumulation is decomposed into a chain of nine Processing Elements (PEs, module systolic_pe) connected in series, each performing exactly one multiply and one add per clock cycle: PE k receives the partial sum from PE (k-1), adds the product of its own pixel and the kernel coefficient at its position, and registers the result to be forwarded to PE (k+1).

Because a new 3x3 window arrives every clock cycle (not every nine cycles), each pixel must be delayed by a number of cycles equal to its chain position minus one, ensuring that every PE always operates on data belonging to the same window as the PE before it. This delay is implemented by the shift_delay module, a generic depth-parameterized shift register. The mode

and data_valid_in signals are delayed following the same principle, sharing an internal shift-register chain. The total pipeline latency of this architecture is 10 clock cycles (nine PE stages plus one final output stage), which reproduces the same final-stage logic as the Adder Tree to guarantee that both architectures produce identical mathematical results.

IV. Algorithms

This section describes the two accumulation algorithms at the algorithmic level, independent of their Verilog implementation, to clarify the structural difference between them before presenting hardware-level implementation details in Section V.

A. Adder Tree Algorithm

Algorithm 1 describes one invocation of the Adder Tree accumulation for a single 3x3 window. The algorithm executes once per clock cycle in steady state, but the effect of a given window is only visible at the output three cycles after the window is presented, due to the three pipeline register stages.

```
Algorithm 1: Adder Tree Accumulation
Input: window[3][3]  // 9 pixels, unsigned 8-bit
Input: mode          // 0 = sharpen, 1 = blur
Output: out_pixel    // 8-bit result

Stage 1 (register, 1 cycle):
for i in 1..3, j in 1..3:
weight[i][j] <- kernel_coefficient(mode, i, j)
product[i][j] <- window[i][j] * weight[i][j]

Stage 2 (register, 1 cycle):
sum <- product[1][1] + product[1][2] + ... + product[3][3]
  // combinational adder tree, 4 addition levels

Stage 3 (register, 1 cycle):
if mode == sharpen:
out_pixel <- clip(sum, 0, 255)
else:  // blur
scaled <- (sum * 114) >> 10   // approximates sum / 9
out_pixel <- clip(scaled, 0, 255)

Total latency: 3 clock cycles
```

B. Systolic Array Algorithm

Algorithm 2 describes the systolic accumulation. Unlike Algorithm 1, the nine terms are not summed in a single step; instead, a running partial sum is threaded through nine sequential stages, one per clock cycle. Because a new window is presented every cycle, the algorithm is

best understood as a steady-state pipeline in which, at any given cycle, PE k is processing the pixel that arrived (k - 1) cycles earlier, while PE (k+1) is processing the pixel that arrived k cycles earlier — one stage behind.

```
Algorithm 2: Systolic Array Accumulation
Input: window[3][3], mode   // one new window per cycle
Output: out_pixel            // 8-bit result, 10 cycles later

// Flatten window into a 9-element sequence in row-major order:
pixel[1..9] <- window[1][1], window[1][2], ..., window[3][3]

// Delay stage: pixel k is delayed by (k - 1) cycles
for k in 1..9:
delayed_pixel[k] <- shift_delay(pixel[k], depth = k - 1)
delayed_mode[k]  <- shift_delay(mode,     depth = k - 1)

// Systolic chain: PE k executes once per cycle
partial_sum[0] <- 0
for k in 1..9:
weight[k]        <- kernel_coefficient(delayed_mode[k], k)
product[k]        <- delayed_pixel[k] * weight[k]
partial_sum[k]   <- register( partial_sum[k-1] + product[k] )
  // exactly ONE multiply and ONE add per PE, per cycle

// Final stage (same logic as Adder Tree Stage 3), 1 more cycle:
if delayed_mode[9] == sharpen:
out_pixel <- clip(partial_sum[9], 0, 255)
else:
scaled <- (partial_sum[9] * 114) >> 10
out_pixel <- clip(scaled, 0, 255)

Total latency: 10 clock cycles (9 PE stages + 1 final stage)
```

The key structural difference between Algorithm 1 and Algorithm 2 is where the O(n) versus O(log n) trade-off is made. Algorithm 1 pays an O(log n) combinational delay once per window, using an O(n) amount of parallel multiply hardware and one combinational adder tree. Algorithm 2 pays a fixed O(1) delay per stage regardless of n, at the cost of O(n) pipeline registers (the shift_delay chains) purely for timing alignment — hardware that has no equivalent in Algorithm 1. This is precisely the resource overhead measured in Section IX.

V. Implementation Details

A. Kernel Coefficients and Fixed-Point Arithmetic

The sharpening kernel uses the coefficient matrix [[0,-1,0],[-1,5,-1],[0,-1,0]], corresponding to a common discrete form of Laplacian-based sharpening [25]. The blur kernel uses all coefficients equal to 1 (an unweighted average of the nine neighboring pixels). Because true division by 9 is

costly on FPGA, the average is approximated by multiplying by the constant 114 and right-shifting by 10 bits (114/1024 ≈ 1/9, with an approximation error of about 0.09%). The accumulator uses a 20-bit signed width, sufficient to hold the largest possible value (9 x 255 = 2295 for blur).

B. Timing Alignment in the Systolic Chain

The required delay for the pixel at chain position k (k = 1 to 9, corresponding to p11 through p33) is (k - 1) clock cycles. This formula guarantees that PE k, upon receiving the partial sum from PE (k-1) at a given cycle, also receives the pixel belonging to the very same original 3x3 window at that cycle — rather than the pixel of a newer window that has since arrived. This was verified directly in simulation: comparing the data_valid_out and o_pixel sequences of both architectures when driven by an identical random data stream confirmed a timing offset of exactly 7 cycles (= 10 - 3) between the two architectures, matching the theoretical pipeline latency difference exactly.

VI. Kernel Matrices and Their Visual Effect on Images

To make the algorithmic difference between the two filter modes concrete, this section presents the two 3x3 kernel matrices used in this design and demonstrates their effect on a synthetic test image, computed with the Python golden model described in Section VII (i.e., the same fixed-point arithmetic as the RTL, guaranteeing the demonstration images are bit-exact with what the hardware produces).

A. Sharpening Kernel

0	-1	0
-1	5	-1
0	-1	0

Matrix 1. 3x3 sharpening kernel (discrete Laplacian-based unsharp mask).

Intuitively, this kernel computes five times the center pixel minus the sum of its four orthogonal neighbors. Where the image is locally flat (all five relevant pixels roughly equal), the result approximates the original pixel value unchanged (5c - 4c = c). Where the center pixel differs sharply from its neighbors — i.e., at an edge — the difference is amplified, pushing the result further from the neighboring pixels than the original image did. This is precisely how a high-pass filter operates: it passes and amplifies high-spatial-frequency content (edges, fine detail) while leaving low-frequency content (flat regions) largely unchanged.

B. Blurring Kernel

1	1	1
1	1	1
1	1	1

Matrix 2. 3x3 blurring kernel (unweighted box average), applied as (sum x 114) >> 10 ≈ sum / 9.

This kernel computes the unweighted average of the pixel and its eight neighbors. Averaging is a low-pass filtering operation: rapid pixel-to-pixel variations (edges, fine texture, noise) are smoothed out because they are averaged together with their surrounding, typically similar, values, while slowly-varying regions (flat areas, gradients) are left almost unchanged, since the average of a locally near-constant neighborhood is close to that same constant.

C. Visual Demonstration

Figure 1 shows a synthetic 128x128 test image containing a mix of large flat regions, a filled circle, a filled rectangle, and a set of closely spaced thin vertical lines — chosen specifically because thin lines and edges are the image content most sensitive to the difference between high-pass and low-pass filtering.

Figure 1. Synthetic input test image (128x128, grayscale).

Figure 2 shows the result of applying the sharpening kernel (Matrix 1). The boundaries of the circle and rectangle appear noticeably more contrasted than in the original, and the thin vertical lines — spaced 8 pixels apart — remain clearly separated and, in fact, appear more distinct than in the original image, since the high-pass response of the kernel amplifies the brightness difference at each line edge.

Figure 2. Output of the sharpening kernel (Matrix 1) on the test image in Figure 1.

Figure 3 shows the result of applying the blurring kernel (Matrix 2) to the same input. The circle and rectangle boundaries are visibly softened, and — most tellingly — the thin vertical lines, which were 8 pixels apart in the original image, are smeared into a much less distinct, near-uniform gray band, since the 3x3 averaging window is wide enough relative to the line spacing to mix each line with its immediate neighbors. This is the clearest visual evidence of low-pass behavior: fine periodic detail is the first content to be lost under averaging, while the large flat regions and the gentle background gradient remain visually almost identical to the original.

Figure 3. Output of the blurring kernel (Matrix 2) on the test image in Figure 1.

Comparing Figures 2 and 3 side by side illustrates, in a single example, the complementary nature of the two kernels used in this design: the sharpening kernel is a high-pass filter that preserves flat regions while amplifying edges and fine detail, whereas the blurring kernel is a low-pass filter that preserves flat regions while suppressing edges and fine detail — the same 3x3 sliding-window convolution mechanism, and the same underlying hardware pipeline (line_buffer, window_3x3, and either accumulation architecture), produces two visually opposite outcomes purely as a function of which coefficient matrix is loaded into the convolution core.

VII. Verification Methodology

Both architectures were functionally verified against an independent Python/NumPy golden reference model that reproduces the RTL's fixed-point arithmetic exactly (identical kernel coefficients, identical accumulator width, identical division-by-9 approximation for blur, identical clipping logic). A set of test images was designed to expose specific classes of bugs: all-zero and all-255 images (overflow/saturation), checkerboard patterns (window read-order errors), single-impulse images (kernel coefficient correctness), and random images.

The verification process uncovered a genuine timing bug in the original cnn_blur.v module (prior to merging into conv_adder_tree): a non-blocking assignment was read back within the same always block before the new value had been committed, causing the blur result to lag its own data_valid_out signal by one extra clock cycle. This bug was confirmed quantitatively by comparing the outputs of the original and merged modules side by side on identical random data: the sharpen channel matched bit-exact 100%, while the blur channel matched 99.98% after compensating for exactly one cycle of offset — confirming the bug hypothesis and simultaneously proving the remainder of the algorithm to be correct.

The verification process also identified an architectural limitation (not a bug): during the period before the line buffer is fully filled after reset (approximately IMG_WIDTH + pipeline latency - 1 cycles), data_valid_out is asserted based on data_valid_in, so the earliest output samples do not reflect a genuine 3x3 window. This offset was measured experimentally at several image widths (8, 16, 64 pixels) and confirmed to match the formula offset ≈ IMG_WIDTH + latency - 1.

VIII. Experimental Setup

Both architectures were synthesized and implemented using Xilinx Vivado, using the same timing constraint file (constraints.xdc) with a 5.500 ns clock period for every measurement, at three image resolutions: 64x64, 128x128, and 256x256 pixels. For each combination (architecture x image size), LUT, flip-flop (FF), and DSP figures were taken from the Report

Utilization after synthesis; WNS (Worst Negative Slack) was taken from the Design Timing Summary after implementation. The maximum operating frequency (Fmax) was computed as:

$$Fmax = 1 / (Clock\ Period - WNS)$$

Since all measurements share the same constrained clock period, the resulting Fmax values are directly comparable across architectures and image sizes.

IX. Experimental Results

A. Resource Utilization and Fmax — Adder Tree Architecture

Image Size	LUT	FF	DSP	WNS (ns)	Fmax (MHz)
64x64	201	259	1	0.150	186.9
128x128	233	323	1	0.116	185.7
256x256	297	451	1	-0.002	181.75

Table I. Resource and timing results for conv_adder_tree.v at three image resolutions (fixed pipeline latency: 3 cycles).

B. Resource Utilization and Fmax — Systolic Array Architecture

Image Size	LUT	FF	DSP	WNS (ns)	Fmax (MHz)
64x64	579	478	1	0.133	186.3
128x128	611	542	1	-0.013	181.4
256x256	671	670	1	0.037	183.05

Table II. Resource and timing results for conv_systolic.v at three image resolutions (fixed pipeline latency: 10 cycles).

C. Resource-Overhead Trend Across Image Sizes

Image Size	LUT Ratio (Systolic/Adder)	FF Ratio (Systolic/Adder)
64x64	2.88x	1.85x
128x128	2.62x	1.68x
256x256	2.26x	1.49x

Table III. LUT/FF overhead ratio of the Systolic Array relative to the Adder Tree, decreasing with image size.

X. Discussion

A. Why Fmax Is Nearly Identical Between the Two Architectures

Tables I and II show that Fmax differs by less than 3% between the two architectures at every image size — far below the theoretical expectation that the Systolic Array should achieve markedly higher Fmax thanks to a per-stage critical path of only one multiply and one add. The cause was identified through source-level analysis: both architectures share an identical,

unmodified final stage for blur mode — a 20-bit multiplication by the constant 114 followed by a comparison (sum x 114 >>> 10). This combinational block, rather than the nine-term accumulation that the Systolic Array was designed to optimize, is very likely the true critical path of both designs. For a 3x3 kernel, the Adder Tree's combinational summation has only four addition levels — already fast enough that it was never the bottleneck to begin with, so shortening it further via the Systolic Array yields no measurable Fmax benefit.

B. Why the Resource Overhead Shrinks with Image Size

Table III shows that the LUT/FF ratio between the Systolic Array and the Adder Tree decreases as the image grows larger (LUT: 2.88x -> 2.26x; FF: 1.85x -> 1.49x). This is explained by the different nature of the two cost sources in the system: the cost of the shift_delay chains — the main source of overhead between the two architectures — depends on kernel size (fixed at nine positions, independent of image size), while the cost of the line_buffer — shared, unchanged, between both architectures — grows linearly with image width. As the image grows larger, the shared cost increasingly dominates the total resource count, diluting the Systolic Array's fixed overhead, so the relative ratio between the two architectures shrinks even though the absolute LUT/FF difference continues to grow.

C. Implications for Architecture Selection

For a 3x3 kernel at the scale tested in this paper, the Adder Tree is the more resource-efficient choice on every measured dimension: fewer LUTs/FFs, lower latency (3 versus 10 cycles), and comparable Fmax. The theoretical advantage of the Systolic Array — a shorter, kernel-size-independent critical path — is expected to become visible only once the kernel is large enough that the Adder Tree's combinational summation (with O(log n) addition levels) exceeds the delay of the final-stage bottleneck that currently dominates both architectures at 3x3 scale.

XI. Limitations

- Resource and Fmax results were obtained via static timing analysis after implementation in Vivado; the design has not yet been deployed and measured on physical FPGA hardware (no real power measurements are available).
- Kernel coefficients are hardcoded per mode (selected via an internal case/mux); runtime configuration through a register interface is not yet supported.
- The system has only been tested on grayscale images; RGB support is not yet implemented.
- The pipeline does not explicitly handle image boundaries: during the period before the line buffer is filled after reset, the earliest output samples do not reflect a genuine 3x3 window, as discussed in Section VII.

- The conclusion regarding the location of the critical path (the final blur-scaling stage) is inferred from source-code structure and overall WNS figures; it has not yet been directly confirmed using Vivado's detailed Critical Path Viewer.

XII. Conclusion and Future Work

This paper presented the design, implementation, verification, and experimental evaluation of two hardware architectures — Adder Tree and Systolic Array — for the accumulation stage of a real-time 3x3 convolution processor on FPGA. Experimental results at three image resolutions show that, for a 3x3 kernel, the Systolic Array does not yet provide a measurable Fmax benefit over the simpler Adder Tree, because the design's true critical path lies in a final stage shared unchanged between both architectures rather than in the accumulation stage. At the same time, the Systolic Array's relative resource overhead was shown to decrease as image size increases, since its fixed, kernel-size-dependent cost is increasingly diluted by the shared, image-size-dependent cost of the line buffer.

Future work includes: extending the comparison to larger kernels (5x5, 7x7) to confirm the hypothesis regarding the conditions under which the Systolic Array provides a genuine Fmax benefit; adding Gaussian and Sobel filter support; supporting runtime-configurable kernels via a register interface; adding RGB image support; deploying and measuring the design on physical FPGA hardware, including power consumption; and adding an AXI4-Stream wrapper for integration into larger SoC/camera-pipeline systems.

References

[1] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "A survey of FPGA-based neural network inference accelerators," ACM Trans. Reconfigurable Technol. Syst., vol. 12, no. 1, pp. 1-26, 2019.

[2] S. Mittal, "A survey of FPGA-based accelerators for convolutional neural networks," Neural Computing and Applications, vol. 32, pp. 1109-1139, 2020.

[3] H. T. Kung, "Why systolic architectures?" IEEE Computer, vol. 15, no. 1, pp. 37-46, Jan. 1982.

[4] H. T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," in Sparse Matrix Proceedings 1978, I. S. Duff and G. W. Stewart, Eds. SIAM, 1979, pp. 256-282.

[5] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA), 2017, pp. 1-12.

[6] A. Dua, Y. Li, and F. Ren, "Systolic-CNN: An OpenCL-defined scalable run-time-flexible FPGA accelerator architecture for accelerating convolutional neural network inference in cloud/edge computing," arXiv:2012.03177, 2020.

[7] "An FPGA based tiled systolic array generator to accelerate CNNs," in Proc. IEEE Conf., 2022, doi: 10.1109/document/9996909.

[8] X. Liu et al., "WinoCNN: Kernel sharing Winograd systolic array for efficient convolutional neural network acceleration on FPGAs," arXiv:2107.04244, 2021.

[9] Y. Wan, J. Chen, X. Yang et al., "DSA-CNN: an FPGA-integrated deformable systolic array for convolutional neural network acceleration," Applied Intelligence, vol. 55, article 65, 2025.

[10] J. Zhang, W. Zhang, G. Luo, X. Wei, Y. Liang, and J. Cong, "Frequency improvement of systolic array-based CNNs on FPGAs," in Proc. IEEE Int. Symp. Circuits and Systems (ISCAS), 2019, pp. 1-4.

[11] "High-frequency systolic array-based transformer accelerator on field programmable gate arrays," Electronics, vol. 12, no. 4, p. 822, 2023.

[12] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[13] H. T. Kung and R. L. Picard, "One-dimensional systolic arrays for multidimensional convolution and resampling," in VLSI for Pattern Recognition and Image Processing, Springer Series in Information Sciences, vol. 13, 1984, pp. 9-24.

[14] "High-throughput systolic array-based accelerator for hybrid transformer-CNN networks," Journal of King Saud University – Computer and Information Sciences, 2024.

[15] "Design space exploration for image processing architectures on FPGA targets," arXiv:1404.3877.

[16] "Memory requirements for sliding window operations in FPGA accelerators," ResearchGate technical figure and accompanying discussion.

[17] "Real-time FPGA based implementation of color image edge detection," ResearchGate, sliding window memory buffer architecture.

[18] "Real-time semantic segmentation on FPGAs for autonomous vehicles with hls4ml," arXiv:2205.07690, 2022.

[19] R. Shi, J. S. J. Wong, and H. K.-H. So, "High-throughput line buffer microarchitecture for arbitrary sized streaming image processing," J. Imaging, vol. 5, no. 3, p. 34, 2019.

[20] "Multi-operand decimal addition by efficient reuse of a binary carry-save adder tree," in Proc. IEEE Conf., 2011, doi: 10.1109/document/5757826.

[21] U. Zimmermann, "Binary adder architectures for cell-based VLSI and their synthesis," Diss. ETH No. 12480, ETH Zurich, 1997.

[22] "LUXOR: An FPGA logic cell architecture for efficient compressor tree implementations," arXiv:2003.03043.

[23] "Unsharp masking," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Unsharp_masking

[24] H. Kwong and J. Liang, "Unsharp masking using center weighted local variance for image sharpening and noise suppression," U.S. Patent 5 081 692, Jan. 1992.

[25] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 2nd ed. Reading, MA: Addison-Wesley, 2002.

[26] "Timing closure techniques for high-performance FPGAs," Number Analytics Technical Blog, 2025.

[27] "Practical timing closure in FPGA and ASIC designs," arXiv:2510.26985.

[28] "SystemVerilog reference verification methodology: RTL," EDN / EE Times, 2006.

[29] "The ultimate guide to FPGA test benches," HardwareBee Technical Guide, 2021.

[30] "TheHuzz: Instruction fuzzing of processors using golden-reference models for finding software-exploitable vulnerabilities," arXiv:2201.09941.